Bayesian Model for March Madness Score Differential Prediction

Eftyghia Kourtelidis

2026-04-04

Bayesian Linear Regression

This notebook predicts tournament point differential using a Bayesian linear regression model fit with brms. The model uses pairwise difference features between Team 1 and Team 2 derived from regular season statistics, with team-level random effects to account for residual team-specific variability.

Features

For each statistic, features are computed as Team 1 minus Team 2, so each row is a vector encoding relative strength between the two teams for that matchup:

- SeedDiff: difference in tournament seed number
- WinPctDiff: difference in regular season win percentage
- ScoreDiffDiff: difference in average regular season point differential
- OffDiff: difference in average points scored (offensive strength)
- DefDiff: difference in average points allowed (defensive strength)
- SeedDiff:WinPctDiff: interaction between seed and win percentage differences

Preprocessing

- Train/test split by chronological year (2008–2022 train, 2023–2025 test) to match the time series split.
- Team assignments (Team 1 vs. Team 2) are randomized to remove any systematic bias from winners always appearing on one side in the raw data.
- All features are standardized using training data only, then the same transform is applied to test years.

Model

A Bayesian linear regression model with Gaussian likelihood:

$$y_i \mid w, \sigma, x_i \sim \mathcal{N}(\mu_i = w^T x_i + u_{T1} + u_{T2}, \sigma^2)$$

where u_{T1}, u_{T2} are team-level random intercepts that account for residual team-specific variability not captured by the observed features.

Priors

$$w_j \sim \mathcal{N}(0,1), \quad \alpha \sim \mathcal{N}(0,5), \quad \sigma \sim \text{Exponential}(1), \quad u_j \sim \mathcal{N}(0,\tau^2)$$

```r
# load libraries
library(brms)
```

```
## Loading required package: Rcpp
```

```
## Loading 'brms' package (version 2.23.0). Useful instructions
## can be found by typing help('brms'). A more detailed introduction
## to the package is available through vignette('brms_overview').

##
## Attaching package: 'brms'

## The following object is masked from 'package:stats':
##
##     ar
```

```r
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.2     v tibble    3.3.1
## v lubridate 1.9.4     v tidyr     1.3.2
## v purrr     1.2.1

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
# load data
teams <- read.csv("Dataset/MTeams.csv")
regular <- read.csv("Dataset/MRegularSeasonCompactResults.csv")
tourney <- read.csv("Dataset/MNCAATourneyCompactResults.csv")
seeds <- read.csv("Dataset/MNCAATourneySeeds.csv")

# ensure IDs are numeric
regular$WTeamID <- as.numeric(regular$WTeamID)
regular$LTeamID <- as.numeric(regular$LTeamID)

seeds$TeamID <- as.numeric(seeds$TeamID)

head(teams)
```

```
##   TeamID    TeamName FirstD1Season LastD1Season
## 1   1101 Abilene Chr          2014         2026
## 2   1102   Air Force          1985         2026
## 3   1103       Akron          1985         2026
## 4   1104     Alabama          1985         2026
## 5   1105 Alabama A&M          2000         2026
## 6   1106  Alabama St          1985         2026
```

```r
head(regular)
```

```
##   Season DayNum WTeamID WScore LTeamID LScore WLoc NumOT
## 1   1985     20    1228     81    1328     64    N     0
## 2   1985     25    1106     77    1354     70    H     0
## 3   1985     25    1112     63    1223     56    H     0
## 4   1985     25    1165     70    1432     54    H     0
## 5   1985     25    1192     86    1447     74    H     0
## 6   1985     25    1218     79    1337     78    H     0
```

```r
head(tourney)
```

```
##   Season DayNum WTeamID WScore LTeamID LScore WLoc NumOT
## 1   1985    136    1116     63    1234     54    N     0
## 2   1985    136    1120     59    1345     58    N     0
## 3   1985    136    1207     68    1250     43    N     0
## 4   1985    136    1229     58    1425     55    N     0
## 5   1985    136    1242     49    1325     38    N     0
## 6   1985    136    1246     66    1449     58    N     0
```

```r
head(seeds)
```

```
##   Season Seed TeamID
## 1   1985  W01   1207
## 2   1985  W02   1210
## 3   1985  W03   1228
## 4   1985  W04   1260
## 5   1985  W05   1374
## 6   1985  W06   1208
```

```r
# build team statistics
# score differential
regular <- regular %>%
  mutate(ScoreDiff = WScore - LScore)

team_stats <- bind_rows(

  # wins
  regular %>%
    group_by(Season, TeamID = WTeamID) %>%
    summarise(
      Games = n(),
      Wins = n(),
      PointsFor = mean(WScore),
      PointsAgainst = mean(LScore),
      ScoreDiff = mean(ScoreDiff),
      .groups = "drop"
    ),

  # losses
  regular %>%
    group_by(Season, TeamID = LTeamID) %>%
    summarise(
      Games = n(),
      Losses = n(),
      PointsFor = mean(LScore),
      PointsAgainst = mean(WScore),
      ScoreDiff = mean(-ScoreDiff),
      .groups = "drop"
    )

) %>%
  group_by(Season, TeamID) %>%
  summarise(
    Games = sum(Games),
```

```r
    Wins = sum(Wins, na.rm = TRUE),
    Losses = sum(Losses, na.rm = TRUE),
    PointsFor = mean(PointsFor),
    PointsAgainst = mean(PointsAgainst),
    ScoreDiff = mean(ScoreDiff),
    .groups = "drop"
  ) %>%
  mutate(
    WinPct = Wins / Games,
    OffStrength = PointsFor,
    DefStrength = PointsAgainst
  )
```

```r
# build tournament dataset
games <- tourney %>%
  mutate(
    POINT_DIFFERENTIAL = WScore - LScore,
    TEAM1_ID = WTeamID,
    TEAM2_ID = LTeamID,
    YEAR = Season
  )

set.seed(1)

# randomize team order to remove winner bias
games <- games %>%
  mutate(random_flip = sample(c(TRUE, FALSE), n(), replace = TRUE)) %>%
  mutate(
    T1 = TEAM1_ID,
    T2 = TEAM2_ID,
    PD = POINT_DIFFERENTIAL
  ) %>%
  mutate(
    TEAM1_ID = ifelse(random_flip, T2, T1),
    TEAM2_ID = ifelse(random_flip, T1, T2),
    POINT_DIFFERENTIAL = ifelse(random_flip, -PD, PD)
  ) %>%
  select(-T1, -T2, -PD, -random_flip)
```

```r
# merge stats
games <- games %>%
  left_join(team_stats,
            by = c("TEAM1_ID" = "TeamID", "YEAR" = "Season")) %>%
  rename(
    WinPct_T1 = WinPct,
    ScoreDiff_T1 = ScoreDiff,
    OffStrength_T1 = OffStrength,
    DefStrength_T1 = DefStrength
  ) %>%

  left_join(team_stats,
            by = c("TEAM2_ID" = "TeamID", "YEAR" = "Season")) %>%
  rename(
    WinPct_T2 = WinPct,
```

```r
    ScoreDiff_T2 = ScoreDiff,
    OffStrength_T2 = OffStrength,
    DefStrength_T2 = DefStrength
  )
```

```r
# seeds
seeds$SeedNum <- as.numeric(substr(seeds$Seed, 2, 3))

games <- games %>%
  left_join(seeds,
            by = c("TEAM1_ID" = "TeamID", "YEAR" = "Season")) %>%
  rename(Seed_T1 = SeedNum) %>%
  left_join(seeds,
            by = c("TEAM2_ID" = "TeamID", "YEAR" = "Season")) %>%
  rename(Seed_T2 = SeedNum)
```

```r
# feature engineering
games <- games %>%
  mutate(
    # handle small NA cases safely
    OffStrength_T1 = ifelse(is.na(OffStrength_T1),
                            median(OffStrength_T1, na.rm = TRUE),
                            OffStrength_T1),
    OffStrength_T2 = ifelse(is.na(OffStrength_T2),
                            median(OffStrength_T2, na.rm = TRUE),
                            OffStrength_T2),
    DefStrength_T1 = ifelse(is.na(DefStrength_T1),
                            median(DefStrength_T1, na.rm = TRUE),
                            DefStrength_T1),
    DefStrength_T2 = ifelse(is.na(DefStrength_T2),
                            median(DefStrength_T2, na.rm = TRUE),
                            DefStrength_T2),

    # core features
    SeedDiff = Seed_T1 - Seed_T2,
    WinPctDiff = WinPct_T1 - WinPct_T2,
    ScoreDiffDiff = ScoreDiff_T1 - ScoreDiff_T2,
    OffDiff = OffStrength_T1 - OffStrength_T2,
    DefDiff = DefStrength_T1 - DefStrength_T2,

    # interaction
    Seed_Win_Interaction = SeedDiff * WinPctDiff,

    # round factor
    Round = as.factor(DayNum)
  )
```

```r
# scale features using training data statistics only to prevent data leakage
# into the test set
games_clean <- games %>%
  filter(
    !is.na(POINT_DIFFERENTIAL),
    !is.na(WinPct_T1),
    !is.na(WinPct_T2),
    !is.na(Seed_T1),
```

```r
    !is.na(Seed_T2)
  ) %>%
  mutate(
    SeedDiff = as.numeric(scale(SeedDiff)),
    WinPctDiff = as.numeric(scale(WinPctDiff)),
    ScoreDiffDiff = as.numeric(scale(ScoreDiffDiff)),
    OffDiff = as.numeric(scale(OffDiff)),
    DefDiff = as.numeric(scale(DefDiff))
  )
```

```r
# train test split
set.seed(1)

train <- games_clean %>% filter(YEAR >= 2008 & YEAR<= 2022)
test <- games_clean %>% filter(YEAR >= 2023 & YEAR <= 2025)
```

```r
# bayesian model
model <- brm(
  POINT_DIFFERENTIAL ~
    SeedDiff +
    WinPctDiff +
    ScoreDiffDiff +
    OffDiff +
    DefDiff +
    SeedDiff:WinPctDiff +
    Round +
    (1 | TEAM1_ID) +
    (1 | TEAM2_ID),
  data = train,
  family = gaussian(),
  prior = c(
    brms::prior(normal(0, 1), class = "b"),
    brms::prior(normal(0, 5), class = "Intercept"),
    brms::prior(exponential(1), class = "sigma")
  ),
  chains = 4,
  iter = 4000,
  control = list(adapt_delta = 0.99)  # reduces divergent transitions
                                      # at the cost of slower sampling
)
```

```
## Compiling Stan program...

## Trying to compile a simple C file

## Running /Library/Frameworks/R.framework/Resources/bin/R CMD SHLIB foo.c
## using C compiler: 'Apple clang version 14.0.3 (clang-1403.0.22.14.1)'
## using SDK: 'MacOSX13.3.sdk'
## clang -arch arm64 -std=gnu2x -I"/Library/Frameworks/R.framework/Resources/include" -DNDEBUG   -I"/Li
## In file included from <built-in>:1:
## In file included from /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library/StanHeade
## In file included from /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library/RcppEigen
## In file included from /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library/RcppEigen
## /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library/RcppEigen/include/Eigen/src/Cor
## #include <cmath>
```

```
##          ^~~~~~
## 1 error generated.
## make: *** [foo.o] Error 1

## Start sampling

##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 0.00013 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 1.3 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 1: Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 1: Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 1: Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 1: Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1: Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 1: Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 1: Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 1: Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 1: Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1: Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 1: Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 1:
## Chain 1:  Elapsed Time: 7.354 seconds (Warm-up)
## Chain 1:                5.485 seconds (Sampling)
## Chain 1:                12.839 seconds (Total)
## Chain 1:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
## Chain 2:
## Chain 2: Gradient evaluation took 4.7e-05 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.47 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 2: Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 2: Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 2: Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 2: Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 2: Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 2: Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 2: Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 2: Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 2: Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 2: Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 2: Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 2:
## Chain 2:  Elapsed Time: 7.507 seconds (Warm-up)
## Chain 2:                5.494 seconds (Sampling)
## Chain 2:                13.001 seconds (Total)
```

```
## Chain 2:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
## Chain 3:
## Chain 3: Gradient evaluation took 4.2e-05 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.42 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:
## Chain 3: Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 3: Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3: Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 3: Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 3: Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 3: Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 3: Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 3: Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 3: Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 3: Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 3: Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 3: Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 3:
## Chain 3:  Elapsed Time: 8.998 seconds (Warm-up)
## Chain 3:                9.968 seconds (Sampling)
## Chain 3:                18.966 seconds (Total)
## Chain 3:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 4.2e-05 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.42 seconds.
## Chain 4: Adjust your expectations accordingly!
## Chain 4:
## Chain 4:
## Chain 4: Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 4: Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 4: Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 4: Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 4: Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 4: Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 4: Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 4: Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4: Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4: Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 4: Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 4: Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 4:
## Chain 4:  Elapsed Time: 7.298 seconds (Warm-up)
## Chain 4:                5.478 seconds (Sampling)
## Chain 4:                12.776 seconds (Total)
## Chain 4:
```

```r
# evaluation for train data
pred <- predict(model, newdata = train)
```

```r
train$PredictedDiff <- pred[,1]

accuracy <- mean(
  (train$PredictedDiff > 0) ==
  (train$POINT_DIFFERENTIAL > 0)
)

rmse <- sqrt(mean((train$POINT_DIFFERENTIAL - train$PredictedDiff)^2))

cat("train accuracy: ", accuracy, "\n")
```

```
## train accuracy:  0.7295259
```

```r
cat("train RMSE: ", rmse, "\n")
```

```
## train RMSE:  10.84756
```

```r
mae <- mean(abs(train$POINT_DIFFERENTIAL - train$PredictedDiff))
cat("train MAE: ", mae, "\n")
```

```
## train MAE:  8.566168
```

```r
# mean predictions
# allow_new_levels = TRUE handles teams in test set not seen during training
pred <- predict(model, newdata = test, allow_new_levels = TRUE)
test$PredictedDiff <- pred[,1]

# posterior samples (uncertainty)
post_preds <- posterior_predict(model, newdata = test, allow_new_levels = TRUE)

# mean (should be very close to predict())
test$PredictedMean <- colMeans(post_preds)

# standard deviation = uncertainty per game
test$PredictedSD <- apply(post_preds, 2, sd)

# accuracy (winner prediction)
accuracy <- mean(
  (test$PredictedDiff > 0) ==
  (test$POINT_DIFFERENTIAL > 0)
)

# RMSE
rmse <- sqrt(mean((test$POINT_DIFFERENTIAL - test$PredictedDiff)^2))

# MAE
mae <- mean(abs(test$POINT_DIFFERENTIAL - test$PredictedDiff))

#summarize uncertainty
avg_uncertainty <- mean(test$PredictedSD)

# print results
cat("test accuracy: ", accuracy, "\n")
```

```
## test accuracy:  0.7164179
```

```r
cat("test RMSE: ", rmse, "\n")
```

```
## test RMSE:  12.24099
```

```r
cat("test MAE: ", mae, "\n")
```

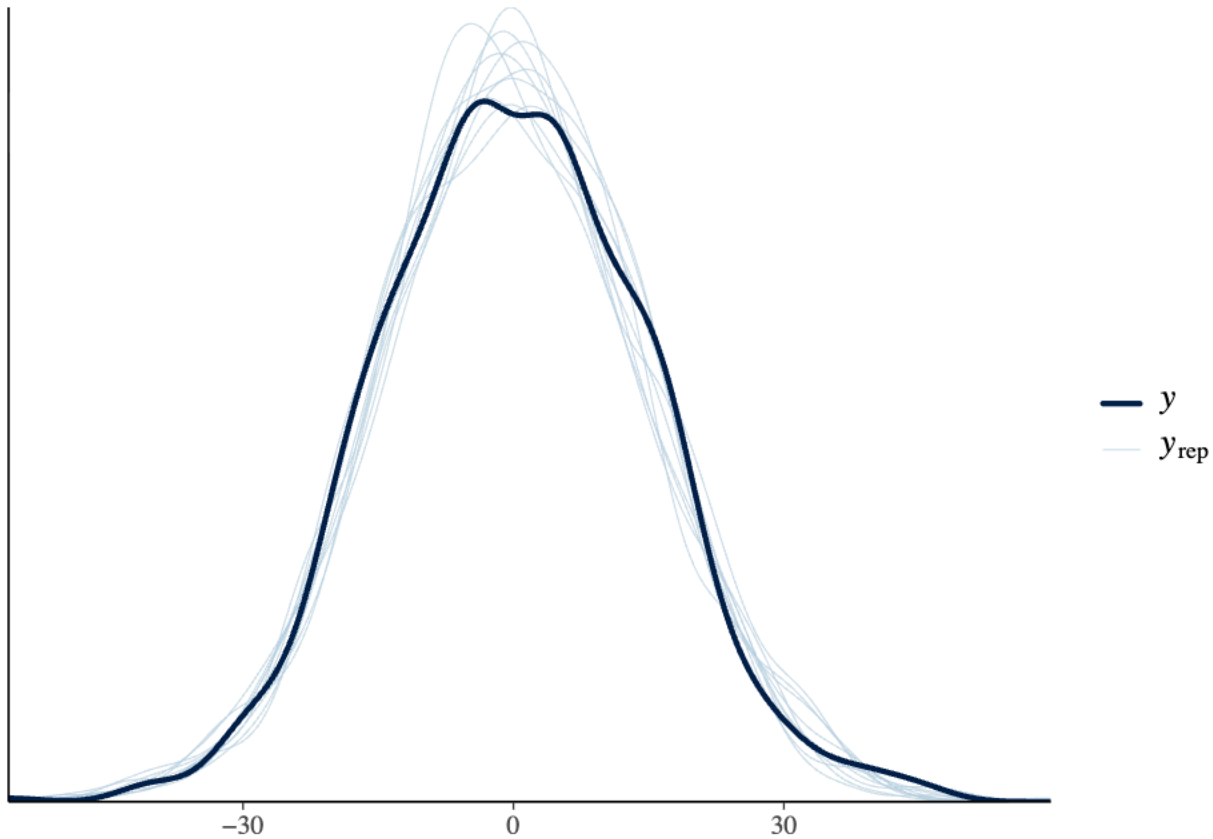
```
## test MAE:  10.0513
```

```r
cat("test avgerage uncertainty: ", avg_uncertainty, "\n")
```

```
## test avgerage uncertainty:  11.82451
```

```r
pp_check(model)
```

```
## Using 10 posterior draws for ppc type 'dens_overlay' by default.
```

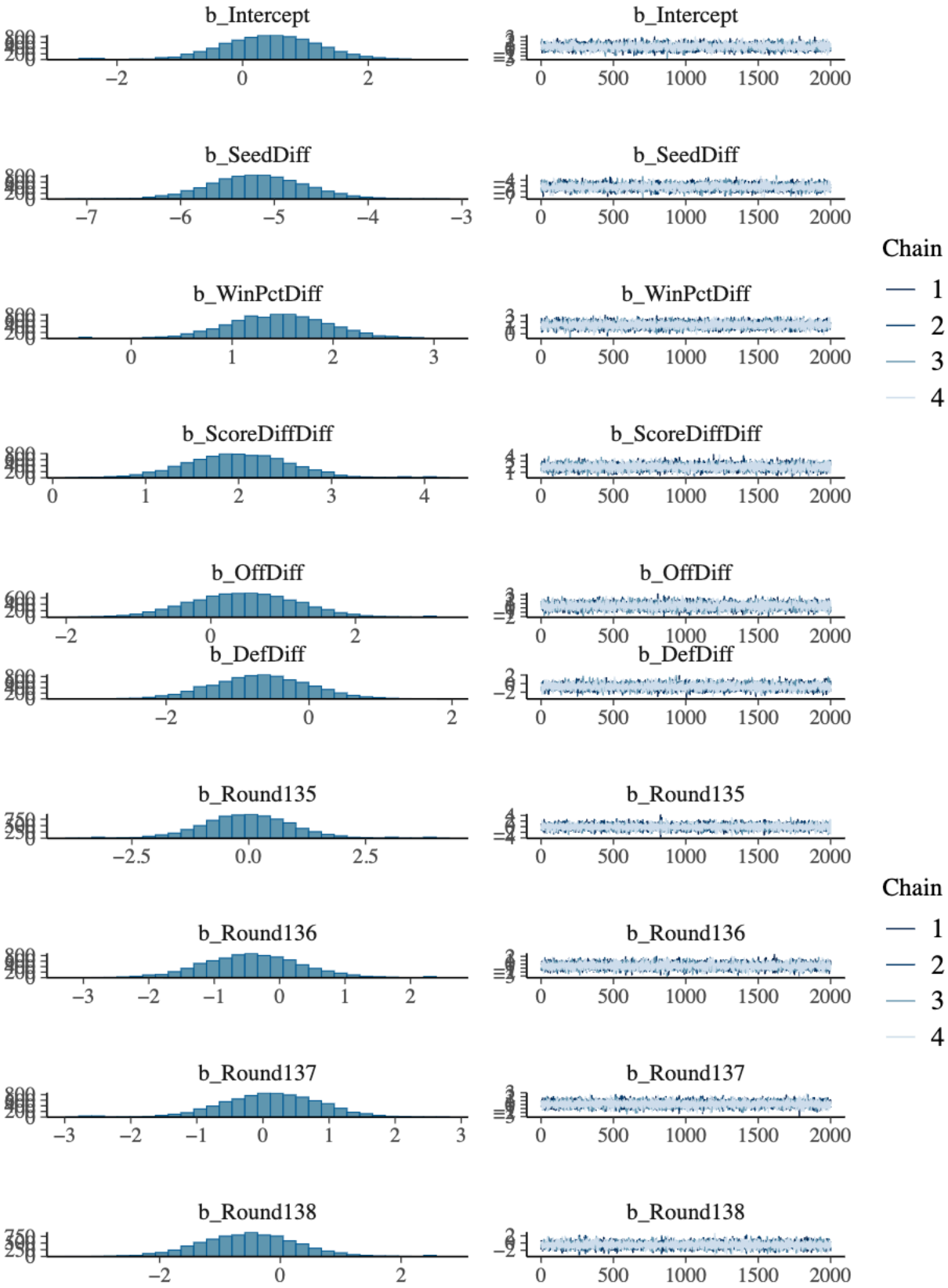


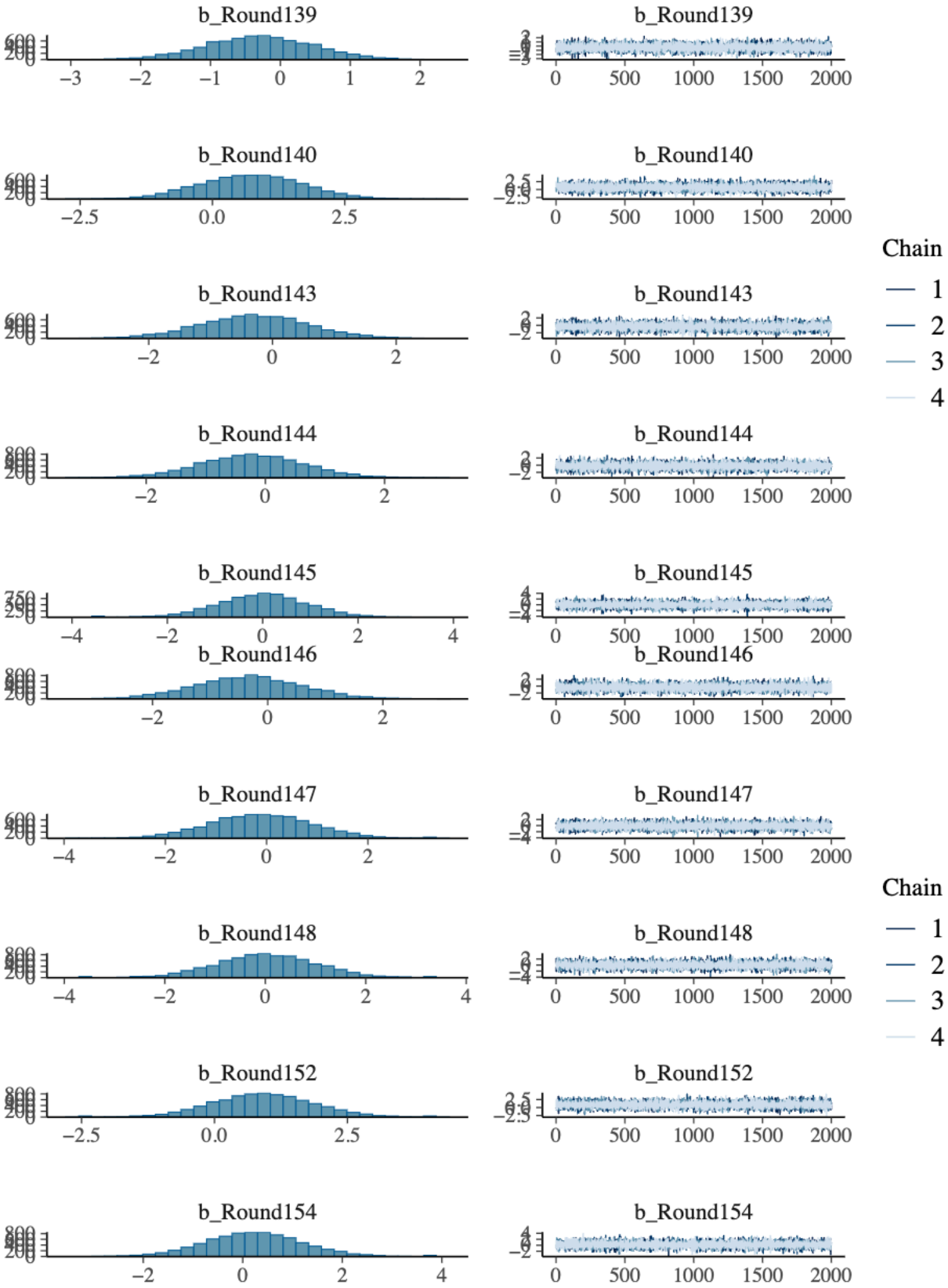

```r
summary(model)
```

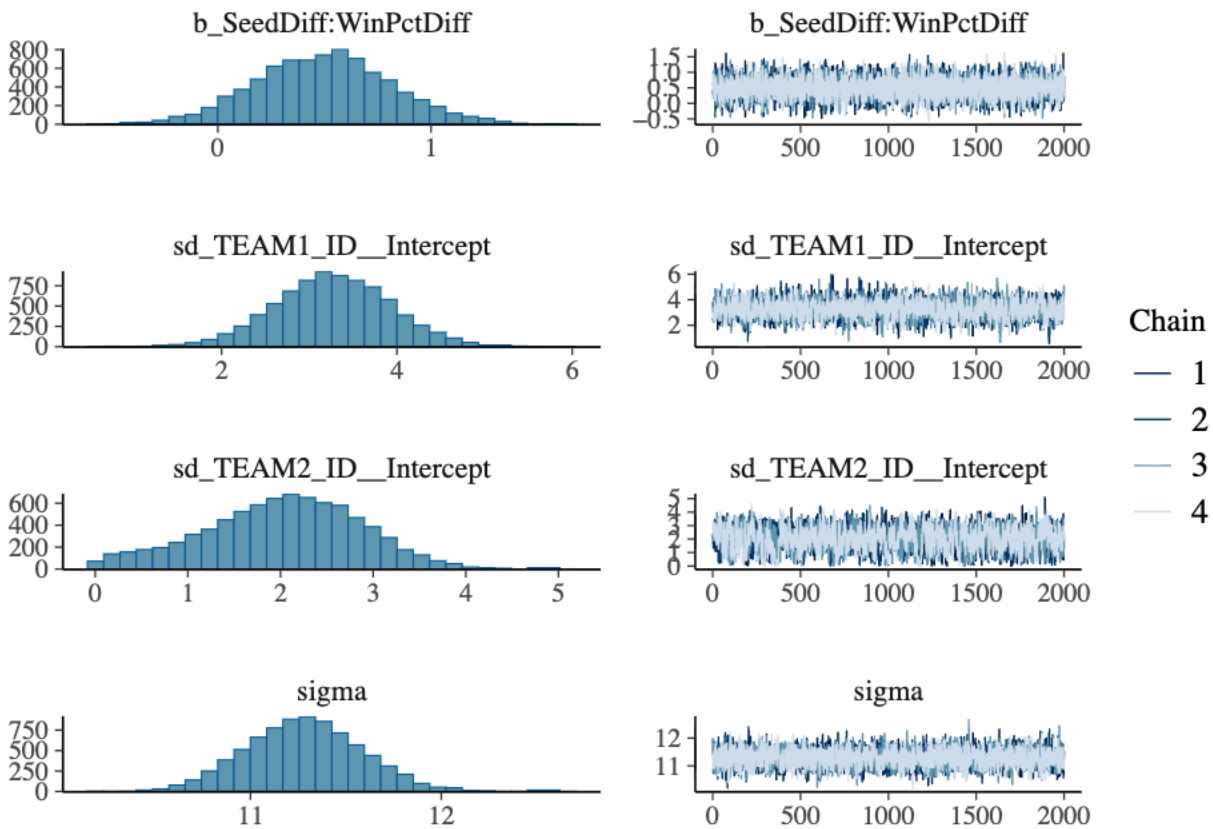
```
##  Family: gaussian
##   Links: mu = identity
## Formula: POINT_DIFFERENTIAL ~ SeedDiff + WinPctDiff + ScoreDiffDiff + OffDiff + DefDiff + SeedDiff:W
##    Data: train (Number of observations: 928)
##   Draws: 4 chains, each with iter = 4000; warmup = 2000; thin = 1;
##          total post-warmup draws = 8000
##
## Multilevel Hyperparameters:
## ~TEAM1_ID (Number of levels: 200)
##               Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## sd(Intercept)     3.27      0.67     1.97     4.57 1.00     2424     3479
##
```

```
## ~TEAM2_ID (Number of levels: 206)
##               Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## sd(Intercept)     2.01      0.84     0.26     3.55 1.00     1472     1546
##
## Regression Coefficients:
##                     Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## Intercept               0.46      0.74    -0.97     1.90 1.00     5607     6218
## SeedDiff               -5.18      0.50    -6.15    -4.18 1.00     6973     6321
## WinPctDiff              1.47      0.48     0.54     2.40 1.00    11067     6585
## ScoreDiffDiff           1.99      0.53     0.95     3.00 1.00     9554     6800
## OffDiff                 0.46      0.72    -0.95     1.85 1.00     8206     7008
## DefDiff                -0.67      0.67    -1.98     0.63 1.00     7936     6557
## Round135               -0.03      0.91    -1.85     1.75 1.00    17378     6122
## Round136               -0.42      0.73    -1.86     1.03 1.00    11811     6321
## Round137                0.14      0.71    -1.24     1.52 1.00    12144     6072
## Round138               -0.52      0.78    -2.05     0.99 1.00    15946     6019
## Round139               -0.30      0.77    -1.80     1.20 1.00    17255     6878
## Round140                0.73      0.95    -1.11     2.60 1.00    14263     6678
## Round143               -0.29      0.87    -2.04     1.42 1.00    18543     5726
## Round144               -0.22      0.87    -1.94     1.47 1.00    14664     5878
## Round145                0.01      0.92    -1.80     1.79 1.00    18704     6391
## Round146               -0.34      0.91    -2.12     1.43 1.00    18031     5691
## Round147               -0.12      0.99    -2.05     1.77 1.00    16236     5656
## Round148                0.02      0.98    -1.89     1.90 1.00    18332     5766
## Round152                0.91      0.93    -0.90     2.74 1.00    16811     5700
## Round154                0.22      0.96    -1.66     2.09 1.00    17714     5543
## SeedDiff:WinPctDiff     0.50      0.32    -0.13     1.14 1.00    17278     6721
##
## Further Distributional Parameters:
##       Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## sigma    11.28      0.30    10.71    11.87 1.00     4852     5924
##
## Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
```

```r
plot(model)
```


b_Intercept
b_SeedDiff
b_WinPctDiff
b_ScoreDiffDiff
b_OffDiff
b_DefDiff
b_Round135
b_Round136
b_Round137
b_Round138
Chain
1
2
3
4


b_Round139
b_Round140
b_Round143
b_Round144
b_Round145
b_Round146
b_Round147
b_Round148
b_Round152
b_Round154
Chain
1
2
3
4

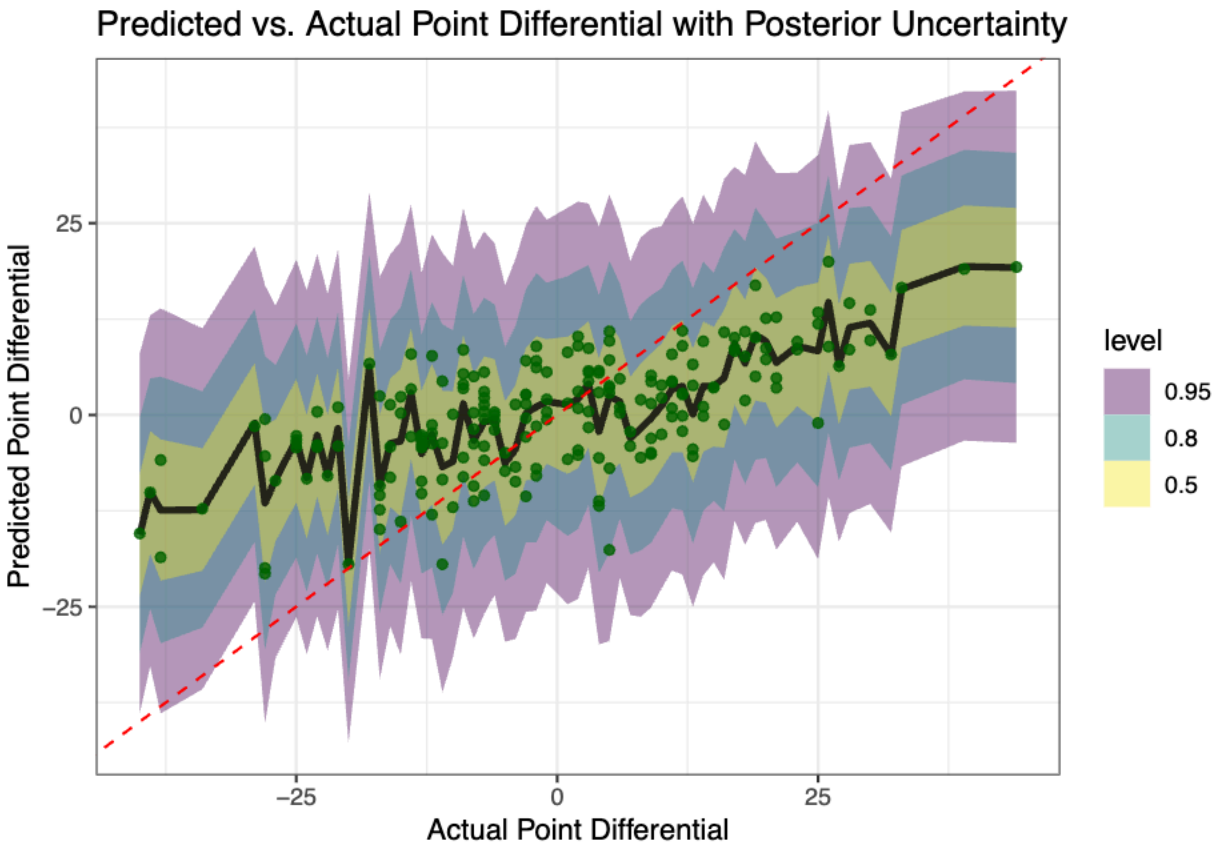



```
library(tidybayes)
```

```
##
## Attaching package: 'tidybayes'

## The following objects are masked from 'package:brms':
##
##     dstudent_t, pstudent_t, qstudent_t, rstudent_t
```

```
library(ggplot2)

# get posterior draws
plot_data <- test %>%
  add_predicted_draws(model, allow_new_levels = TRUE)

# plot
ggplot(plot_data, aes(x = POINT_DIFFERENTIAL, y = .prediction)) +

  # uncertainty bands
  stat_lineribbon(.width = c(0.95, 0.80, 0.50), alpha = 0.4) +

  # actual data points
  geom_point(data = test, aes(x = POINT_DIFFERENTIAL, y = PredictedMean),
             color = "darkgreen", alpha = 0.8) +

  # ideal line (perfect predictions)
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +

  labs(
```

```
    x = "Actual Point Differential",
    y = "Predicted Point Differential",
    title = "Predicted vs. Actual Point Differential with Posterior Uncertainty"
  ) +

  theme_bw()
```




```
ggsave("bayesian_predictions.png", width = 7, height = 5, dpi = 300)
```

```
library(dplyr)

# get train predictions
train_preds <- posterior_predict(model, newdata = train, allow_new_levels = TRUE)
train$PredictedMean <- colMeans(train_preds)

# build train predictions dataframe
train_predictions <- train %>%
  left_join(teams %>% select(TeamID, TeamName) %>% rename(TEAM1_NAME = TeamName),
            by = c("TEAM1_ID" = "TeamID")) %>%
  left_join(teams %>% select(TeamID, TeamName) %>% rename(TEAM2_NAME = TeamName),
            by = c("TEAM2_ID" = "TeamID")) %>%
  mutate(
    SPLIT = "train",
    PRED_MARGIN = PredictedMean,
    ABS_ERROR = abs(POINT_DIFFERENTIAL - PredictedMean)
  ) %>%
  select(
```

```r
    YEAR,
    CURRENT_ROUND = DayNum,
    TEAM1 = TEAM1_NAME,
    TEAM2 = TEAM2_NAME,
    POINT_DIFFERENTIAL,
    SPLIT,
    PRED_MARGIN,
    ABS_ERROR
  )

# build test predictions dataframe
test_predictions <- test %>%
  left_join(teams %>% select(TeamID, TeamName) %>% rename(TEAM1_NAME = TeamName),
            by = c("TEAM1_ID" = "TeamID")) %>%
  left_join(teams %>% select(TeamID, TeamName) %>% rename(TEAM2_NAME = TeamName),
            by = c("TEAM2_ID" = "TeamID")) %>%
  mutate(
    SPLIT = "test",
    PRED_MARGIN = PredictedMean,
    ABS_ERROR = abs(POINT_DIFFERENTIAL - PredictedMean)
  ) %>%
  select(
    YEAR,
    CURRENT_ROUND = DayNum,
    TEAM1 = TEAM1_NAME,
    TEAM2 = TEAM2_NAME,
    POINT_DIFFERENTIAL,
    SPLIT,
    PRED_MARGIN,
    ABS_ERROR
  )

# combine and write to predictions folder
bayesian_predictions <- bind_rows(train_predictions, test_predictions)
write.csv(bayesian_predictions, "Predictions/bayesian_predictions.csv", row.names = FALSE)
```